CS23532 — COMPUTER NETWORKS

LABORATORY EXPERIMENT OUTPUTS

Compiled from the uploaded Computer Networks Lab Manual

Experiments 1–15

Note: Outputs that depend on live devices, PCAP files, server logs or local network conditions are presented as illustrative sample outputs. They are formatted to resemble terminal/tool output but are not claimed to be actual captures from your computer.

Experiment 1: Study of Network Commands in Linux and Windows

OUTPUT

Windows command output

```
C:\> hostname
DESKTOP-PC01

C:\> ipconfig
Ethernet adapter Ethernet:
   IPv4 Address. . . . . . . . . . . : 192.168.1.10
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 192.168.1.1

C:\> ping 192.168.1.1
Pinging 192.168.1.1 with 32 bytes of data:
Reply from 192.168.1.1: bytes=32 time<1ms TTL=64
Reply from 192.168.1.1: bytes=32 time<1ms TTL=64
Reply from 192.168.1.1: bytes=32 time<1ms TTL=64
Reply from 192.168.1.1: bytes=32 time<1ms TTL=64

Ping statistics for 192.168.1.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
```

Linux command output

```
$ ip address show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP>
    inet 192.168.1.100/24 brd 192.168.1.255 scope global eth0

$ ip route show
default via 192.168.1.1 dev eth0
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.100

$ ping -c 4 192.168.1.1
4 packets transmitted, 4 received, 0% packet loss
rtt min/avg/max/mdev = 0.421/0.563/0.701/0.101 ms
```

Result: The experiment output format and expected observations are documented above. Confirm the displayed values against the actual lab run before submission.

Experiment 2: Study of Network Cables and Crimping

OUTPUT

Observed cable wiring

```
Straight-through cable: T568B at both ends
Pin   T568B wire colour
1     White/Orange
2     Orange
3     White/Green
4     Blue
5     White/Blue
6     Green
7     White/Brown
8     Brown

Crossover cable: T568A at one end and T568B at the other end.
Continuity tester result: Pins 1–8 tested in sequence; all eight conductors indicated continuity.
```

Result observation

```
The UTP, STP, coaxial and fibre-optic cable types were studied. Straight-through and crossover Ethernet cabl
```

Result: The experiment output format and expected observations are documented above. Confirm the displayed values against the actual lab run before submission.

Experiment 3: Cisco Packet Tracer: Hub, Switch and Combined Network

OUTPUT

Scenario 1 — Hub

```
Packet Tracer Simulation:
Source: PC0 (192.168.1.1)
Destination: PC2 (192.168.1.3)
Event: ICMP Echo Request
Observation: Hub repeats the incoming frame to all active ports.
PC1: Frame received, destination mismatch
PC2: Frame received, destination match
PC3: Frame received, destination mismatch
PDU Status: Successful
```

Scenario 2 — Switch

```
Packet Tracer Simulation:
Source: PC0 (192.168.1.1)
Destination: PC2 (192.168.1.3)
Event: ICMP Echo Request / Echo Reply
Observation: Switch learns source MAC address. After MAC learning,
the frame is forwarded through the destination port.
PDU Status: Successful
```

Scenario 3 — Hub and switch

```
Packet Tracer Simulation:
PC0 -> Switch0 -> Hub0 -> PC1, PC2, PC3
The switch forwards the frame toward the hub; the hub repeats it
to connected hosts. PC2 accepts the packet; PC1 and PC3 discard it.
PDU Status: Successful
```

Result: The experiment output format and expected observations are documented above. Confirm the displayed values against the actual lab run before submission.

Experiment 4: Setup and Configure a LAN Using a Switch

OUTPUT

IP configuration

```
Device   IPv4 Address   Subnet Mask
PC0      10.1.1.1       255.0.0.0
PC1      10.1.1.2       255.0.0.0
PC2      10.1.1.3       255.0.0.0
PC3      10.1.1.4       255.0.0.0

C:\> ping 10.1.1.2
Reply from 10.1.1.2: bytes=32 time<1ms TTL=128
Reply from 10.1.1.2: bytes=32 time<1ms TTL=128
Reply from 10.1.1.2: bytes=32 time<1ms TTL=128
Reply from 10.1.1.2: bytes=32 time<1ms TTL=128

Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
```

File-sharing verification

```
\\10.1.1.1
Shared folder: LabShare
Test file: network_test.txt
Verification: Shared folder listed and test file opened successfully.
Note: This is a sample expected output; Windows permissions and sharing
settings must be configured on the actual PCs.
```

Result: The experiment output format and expected observations are documented above. Confirm the displayed values against the actual lab run before submission.

Experiment 5: Packet Capture and Protocol Analysis Using Wireshark

OUTPUT

Display filter: icmp

```
No.   Time     Source          Destination     Protocol Length Info
15    2.345    192.168.1.10    8.8.8.8         ICMP     74     Echo (ping) request
16    2.356    8.8.8.8         192.168.1.10    ICMP     74     Echo (ping) reply
17    3.347    192.168.1.10    8.8.8.8         ICMP     74     Echo (ping) request
18    3.359    8.8.8.8         192.168.1.10    ICMP     74     Echo (ping) reply

Packets displayed: 4
Display filter: icmp
```

Packet details

```
Frame: Ethernet II, 74 bytes on wire
Ethernet II
    Source: 00:1A:2B:3C:4D:5E
    Destination: 00:AA:BB:CC:DD:EE
Internet Protocol Version 4
    Source Address: 192.168.1.10
    Destination Address: 8.8.8.8
Internet Control Message Protocol
    Type: 8 (Echo request)
    Code: 0
```

Result: The experiment output format and expected observations are documented above. Confirm the displayed values against the actual lab run before submission.

Experiment 6: Building an Anonymous FTP Login Checker Using ftplib

OUTPUT

Illustrative terminal output

```
C:\Lab> python ftp_check.py
Target FTP host: ftp.example.test
Connecting to ftp.example.test...
Connection established.
Attempting anonymous login...
[!] Server response: 530 Login incorrect.
[-] FTP anonymous login was not permitted.
Connection closed.
```

Note

FTP access and server responses vary. This example shows a realistic denied-login response; run the experiment only against a server you own or are authorized to test.

Result: The experiment output format and expected observations are documented above. Confirm the displayed values against the actual lab run before submission.

Experiment 7: Develop a Simple Calculator Using XML-RPC

OUTPUT

Sample program output

```
------ XML-RPC Calculator ------
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit

Enter your Choice: 1
Enter First Number: 25
Enter Second Number: 15
Result = 40

------ XML-RPC Calculator ------
Enter your Choice: 3
Enter First Number: 8
Enter Second Number: 6
Result = 48

------ XML-RPC Calculator ------
Enter your Choice: 4
Enter First Number: 15
Enter Second Number: 0
Result = Error! Division by Zero

------ XML-RPC Calculator ------
Enter your Choice: 5
Calculator Closed.
```

Result: The experiment output format and expected observations are documented above. Confirm the displayed values against the actual lab run before submission.

Experiment 8: Examine Network Address Translation (NAT) Using Cisco Packet Tracer

OUTPUT

PC0 ping output

```
PC> ping 203.0.113.10

Pinging 203.0.113.10 with 32 bytes of data:
Reply from 203.0.113.10: bytes=32 time<1ms TTL=127
Reply from 203.0.113.10: bytes=32 time<1ms TTL=127
Reply from 203.0.113.10: bytes=32 time<1ms TTL=127
Reply from 203.0.113.10: bytes=32 time<1ms TTL=127

Ping statistics:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
```

Router NAT translation table

```
Router# show ip nat translations
Pro  Inside global       Inside local        Outside local      Outside global
icmp 203.0.113.1:1       192.168.1.10:1      203.0.113.10:1     203.0.113.10:1

Router# show ip nat statistics
Total active translations: 1
Inside interfaces:  GigabitEthernet0/0
Outside interfaces: GigabitEthernet0/1
Hits: 5  Misses: 0
```

Result: The experiment output format and expected observations are documented above. Confirm the displayed values against the actual lab run before submission.

Experiment 9: TCP Socket Client–Server Command Demonstration

OUTPUT

Server terminal

```
C:\Lab> python server.py
Server is listening on 127.0.0.1 5000
Connected by: ('127.0.0.1', 54321)
Client Command: hello
Client Command: time
Client Command: date
Client Command: test
Client Command: exit
```

Client terminal

```
C:\Lab> python client.py
Connected Successfully
Available Commands
1. hello
2. time
3. date
4. exit

Enter Command: hello
Server: Hello from Server!

Enter Command: time
Server: 10:35:42

Enter Command: date
Server: 09-07-2026

Enter Command: test
Server: Invalid Command

Enter Command: exit
Server: Connection Closed
```

Safety note

```
The supplied lab describes a restricted command/response TCP client-server demo, not a general-purpose opera
```

Result: The experiment output format and expected observations are documented above. Confirm the displayed values against the actual lab run before submission.

Experiment 10: Configure a Topology with One Router, Two Switches and Four PCs

OUTPUT

Router interface status

```
R1# show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0      192.168.1.1     YES manual up                    up
GigabitEthernet0/1      192.168.2.1     YES manual up                    up
Vlan1                   unassigned      YES unset  administratively down down
```

Connectivity test

```
PC0> ping 192.168.2.10
Pinging 192.168.2.10 with 32 bytes of data:
Reply from 192.168.2.10: bytes=32 time<1ms TTL=127
Reply from 192.168.2.10: bytes=32 time<1ms TTL=127
Reply from 192.168.2.10: bytes=32 time<1ms TTL=127
Reply from 192.168.2.10: bytes=32 time<1ms TTL=127

Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
```

Result: The experiment output format and expected observations are documented above. Confirm the displayed values against the actual lab run before submission.

Experiment 11: Study of Socket Programming in Python

OUTPUT

Server output

```
C:\Python> python server.py
Server is waiting for connection...
Client connected: ('127.0.0.1', 54321)
Message from client: Hello Server!
```

Client output

```
C:\Python> python client.py
Message from server: Hello Client! Message received successfully.
```

Result: The experiment output format and expected observations are documented above. Confirm the displayed values against the actual lab run before submission.

Experiment 12: Network Forensics Using PcapXray

OUTPUT

Sample analysis summary

```
PcapXray - PCAP Analysis Report
Input file       : ipv4frags.pcap
File type        : PCAP
Analysis status  : Completed (sample report format)

Summary
--------------------------------
Packets analyzed : 24
Unique hosts     : 2
Protocols found  : IPv4, ICMP
Capture duration : 0.018 seconds

Host overview
192.168.0.10  <->  192.168.0.20
Observed traffic: IPv4 fragment packets

Report sections
[✓] Packet summary
[✓] Host communication graph
[✓] Protocol distribution
[✓] Export report
```

Note

Packet count, host addresses and duration above are illustrative sample values to show the expected report layout. Actual values must be copied from the PCAP file you analyze.

Result: The experiment output format and expected observations are documented above. Confirm the displayed values against the actual lab run before submission.

Experiment 13: Analyze Server Logs Using Webalizer

OUTPUT

Example Webalizer summary

```
Webalizer Web Statistics
Report generated: October 2026
Site: localhost

Summary Period: October 2026
Total Hits:             1,284
Total Files:              842
Total Pages:              316
Total Visits:              97
Total KBytes:           4,582
Total Unique Sites:        42
Total Unique URLs:         31

Monthly Statistics
Month       Hits   Files   Pages   Visits
Oct 2026    1284    842     316      97

Top URLs
/                  248 hits
/index.html        191 hits
/images/logo.png   102 hits

Output directory: C:\xampp\htdocs\webalizer
```

Note

These are illustrative statistics. Webalizer's actual totals are generated from the access.log file and will differ according to the log contents.

Result: The experiment output format and expected observations are documented above. Confirm the displayed values against the actual lab run before submission.

Experiment 14: Develop a Customized Ping Command and Measure RTT

OUTPUT

Program 1 — connectivity test

```
C:\Lab> python custom_ping.py
Reply from google.com: time=18.52 ms
Reply from google.com: time=17.84 ms
Reply from google.com: time=19.03 ms
Reply from google.com: time=18.26 ms
```

Program 2 — RTT statistics

```
Pinging google.com with TCP packets:

Reply 1: time = 18.24 ms
Reply 2: time = 17.81 ms
Reply 3: time = 19.15 ms
Reply 4: time = 18.56 ms

--- Ping Statistics ---
Packets Sent     : 4
Packets Received : 4
Packets Lost     : 0
Minimum RTT      : 17.81 ms
Maximum RTT      : 19.15 ms
Average RTT      : 18.44 ms
```

Note

Round-trip times are examples only. Network load, DNS resolution, server availability and firewall settings affect actual measurements.

Result: The experiment output format and expected observations are documented above. Confirm the displayed values against the actual lab run before submission.

Experiment 15: Implement Packet Sniffing Using Raw Sockets

OUTPUT

Illustrative packet capture output

```
C:\Lab> python packet_sniffer.py
Starting packet capture...
Capturing 5 packets (sample output)

Packet 1
Source MAC      : 00:1A:2B:3C:4D:5E
Destination MAC : FF:FF:FF:FF:FF:FF
Protocol        : ARP
Frame Length    : 60 bytes

Packet 2
Source MAC      : 00:1A:2B:3C:4D:5E
Destination MAC : 00:AA:BB:CC:DD:EE
Protocol        : IPv4
Frame Length    : 74 bytes

Packet 3
Source MAC      : 00:AA:BB:CC:DD:EE
Destination MAC : 00:1A:2B:3C:4D:5E
Protocol        : IPv4
Frame Length    : 66 bytes

Packet 4
Source MAC      : 00:1A:2B:3C:4D:5E
Destination MAC : 00:AA:BB:CC:DD:EE
Protocol        : IPv4
Frame Length    : 1514 bytes

Packet 5
Source MAC      : 00:AA:BB:CC:DD:EE
Destination MAC : 00:1A:2B:3C:4D:5E
Protocol        : IPv4
Frame Length    : 98 bytes

Capture completed. Total packets processed: 5
```

Note

```
MAC addresses, protocols and frame lengths are illustrative. Raw packet capture requires suitable privileges
```

Result: The experiment output format and expected observations are documented above. Confirm the displayed values against the actual lab run before submission.